ISLAMIC UNIVERSITY OF TECHNOLOGY

Organization of Islamic Cooperation

Name:	Antara Arifa Mullick
ID:	210042132
Program:	SWE
Department:	CSE
Subject:	Artificial Intelligence
Subject Code:	CSE 4618
Title:	Uninformed Search
Date of Submission:	4th June, 2025

1 Introduction

In this lab, we implemented uninformed search algorithms to solve maze navigation problems for a Pacman agent. The primary objective was to develop and compare three fundamental search strategies: Depth-First Search (DFS), Breadth-First Search (BFS), and Uniform Cost Search (UCS). These algorithms enable Pacman to find optimal or near-optimal paths through various maze configurations while demonstrating different search behaviors and performance characteristics.

2 Problem Analysis

The maze navigation problem involves finding a path from an initial state to a goal state in a grid-based environment. The challenges include:

- Avoiding cycles and redundant exploration of states
- Managing memory usage efficiently while maintaining completeness
- Handling varying path costs in different maze configurations
- Ensuring optimal or suboptimal solutions based on algorithm characteristics

Each algorithm addresses these challenges differently, showcasing the trade-offs between optimality, completeness, time complexity, and space complexity in uninformed search strategies.

3 Solution Explanation

Three distinct search algorithms were implemented, each utilizing different data structures and exploration strategies to navigate the maze environment.

3.1 Depth-First Search (DFS)

The DFS implementation employs a Last-In-First-Out (LIFO) strategy using a stack data structure. This approach explores paths as deeply as possible before backtracking, prioritizing depth over breadth in the search process.

Key implementation features:

- Uses a stack to maintain the search frontier
- Maintains a visited set to prevent cycles
- Marks states as visited only after popping from the stack
- Constructs the action sequence incrementally

```
def depthFirstSearch(problem):
    from util import Stack

    stack = Stack()
    start = problem.getStartState()
    stack.push((start, []))
```

```
    visited = set()

    while not stack.isEmpty():
        state, actions = stack.pop()

        if problem.isGoalState(state):
            return actions

        if state not in visited:
            visited.add(state)

            for nextState, action, cost in problem.getSuccessors(state)
    :
                if nextState not in visited:
                    stack.push((nextState, actions + [action]))

    return []
```

Listing 1: Depth-First Search Implementation

3.2 Breadth-First Search (BFS)

The BFS implementation utilizes a First-In-First-Out (FIFO) strategy with a queue data structure. This systematic approach explores all nodes at the current depth level before proceeding to deeper levels, guaranteeing the shortest path in terms of number of steps.

Key implementation features:

- Uses a queue to maintain the search frontier
- Implements early visited marking to prevent redundant enqueueing
- Ensures optimal solutions for uniform-cost problems
- Maintains systematic level-by-level exploration

```
def breadthFirstSearch(problem):
    from util import Queue

    queue = Queue()
    start = problem.getStartState()
    queue.push((start, []))
    visited = set([start])

    while not queue.isEmpty():
        state, actions = queue.pop()

        if problem.isGoalState(state):
            return actions

        for nextState, action, cost in problem.getSuccessors(state):
            if nextState not in visited:
                visited.add(nextState)
                queue.push((nextState, actions + [action]))

    return []
```

Listing 2: Breadth-First Search Implementation

3.3 Uniform Cost Search (UCS)

The UCS implementation employs a priority queue to explore nodes based on cumulative path cost. This approach guarantees optimal solutions by always expanding the lowest-cost path first, making it particularly effective for problems with non-uniform step costs.

Key implementation features:

- Uses a priority queue ordered by cumulative path cost
- Maintains a cost dictionary to track the best known cost to each state
- Implements cost-based pruning to avoid suboptimal paths
- Handles multiple paths to the same state efficiently

```
def uniformCostSearch(problem):
    from util import PriorityQueue

    pq = PriorityQueue()
    start_state = problem.getStartState()
    pq.push((start_state, [], 0), 0)
    visited = {}

    while not pq.isEmpty():
        state, actions, cost = pq.pop()

        if problem.isGoalState(state):
            return actions

        if state not in visited or cost < visited[state]:
            visited[state] = cost

            for nextState, action, stepCost in problem.getSuccessors(
    state):
                cumulative_cost = cost + stepCost
                priority = cumulative_cost

                if nextState not in visited or cumulative_cost <
    visited.get(nextState, float('inf')):
                    pq.push((nextState, actions + [action],
    cumulative_cost), priority)

    return []
```

Listing 3: Uniform Cost Search Implementation

4 Complexity Analysis

Depth-First Search (DFS):

- Time Complexity: $O(b^m)$, where b is the branching factor and m is the maximum depth
- Space Complexity: $O(bm)$, linear space requirement due to stack storage
- Memory efficient but potentially non-optimal solutions

Breadth-First Search (BFS):

- Time Complexity: $O(b^d)$, where d is the depth of the shallowest solution
- Space Complexity: $O(b^d)$, exponential space due to queue storage
- Optimal for uniform costs but high memory consumption

Uniform Cost Search (UCS):

- Time Complexity: $O(b^{C^*/\epsilon})$, where C^* is optimal cost and ϵ is minimum step cost
- Space Complexity: $O(b^{C^*/\epsilon})$, varies with cost structure
- Optimal solutions guaranteed but potentially expensive for uniform costs

5 Findings and Insights

DFS Performance:

- Demonstrated fast execution in linear or narrow maze configurations
- Often produced suboptimal solutions due to depth-first exploration
- Showed excellent memory efficiency, particularly beneficial for deep mazes
- Vulnerable to infinite loops in cyclic environments without proper cycle detection

BFS Performance:

- Consistently delivered optimal solutions in terms of step count
- Required significantly more memory due to frontier expansion
- Performed exceptionally well in mazes with uniform step costs
- Showed systematic exploration pattern ensuring completeness

UCS Performance:

- Provided optimal cost-based solutions regardless of step cost variation
- Performed similarly to BFS in uniform-cost environments
- Demonstrated superior adaptability in mazes with varying path costs
- Required additional computational overhead for cost tracking and priority management

6 Implementation Challenges and Solutions

Challenge 1: Cycle Prevention
The initial implementation faced issues with revisiting states, leading to infinite loops. This was resolved by implementing proper visited state tracking, with different strategies for each algorithm based on their characteristics.

Challenge 2: BFS Optimization
Early implementation allowed multiple enqueueing of the same state. The solution involved marking states as visited immediately upon enqueueing rather than upon dequeuing, significantly improving efficiency.

Challenge 3: UCS Cost Management
Managing multiple paths to the same state with different costs required careful implementation. The solution involved maintaining a cost dictionary and implementing proper cost comparison logic to ensure optimal path selection.

Challenge 4: Action Sequence Construction
Ensuring proper action sequence reconstruction without recursion required careful state management. This was achieved by maintaining complete path information in each frontier node.

7 Hyperparameter Exploration

While uninformed search algorithms have fewer traditional hyperparameters compared to machine learning algorithms, several structural and implementation parameters significantly impact their behavior and performance:

Data Structure Selection: The choice of underlying data structure fundamentally defines the search strategy:

- **Stack (DFS):** LIFO behavior leads to depth-first exploration, resulting in lower memory usage but potentially suboptimal solutions
- **Queue (BFS):** FIFO behavior ensures level-by-level exploration, guaranteeing shortest paths in uniform-cost scenarios but requiring exponential memory
- **Priority Queue (UCS):** Cost-ordered exploration provides optimal solutions but introduces computational overhead for priority management

Visited State Management Strategy: The timing of marking states as visited significantly affects algorithm efficiency:

- **Early Marking (BFS Implementation):** States are marked visited upon enqueueing, preventing duplicate entries in the frontier and reducing memory usage
- **Late Marking (DFS Implementation):** States are marked visited upon expansion, allowing multiple frontier entries but ensuring proper exploration order

Cost Handling in UCS: The strategy for managing multiple paths to the same state affects optimality guarantees:

- **Dictionary-based Tracking:** Our implementation uses a cost dictionary to track the best known cost to each state, enabling efficient pruning of suboptimal paths
- **Priority Queue Management:** Careful handling of duplicate states in the priority queue ensures optimal solution discovery while maintaining efficiency

8 Algorithm Comparison and Trade-offs

The three algorithms demonstrate fundamental trade-offs in search strategy design:

Optimality vs. Efficiency: DFS provides fast solutions but sacrifices optimality, while BFS and UCS guarantee optimal solutions at higher computational cost.

Memory vs. Time: DFS minimizes memory usage but may require more time due to suboptimal path exploration, while BFS trades memory for systematic exploration.

Adaptability vs. Simplicity: UCS provides maximum adaptability to varying cost structures but requires more complex implementation compared to DFS and BFS.

9 Conclusion

This lab successfully demonstrated the implementation and comparison of three fundamental uninformed search algorithms. The results highlight how different data structures and exploration strategies directly influence algorithm behavior and performance characteristics. It reinforced the importance of understanding algorithm trade-offs and selecting appropriate search strategies based on problem characteristics and constraints.